

Deep Learning 1, lecture 4

Convolutional Neural Network (CNN) architectures

Foma Shipilov

LeNet

- **Convolutions** and **Subsampling** (i.e. **Pooling**) to extract features
- **Sigmoid** activation function
- **Fully-connected network** as a classification head
- Works with fixed image size (!)

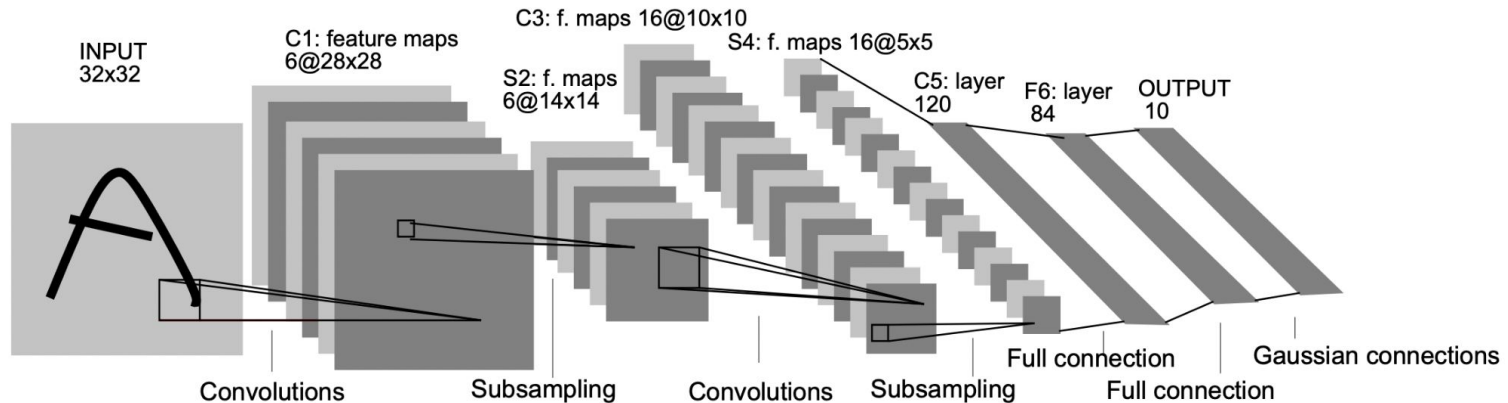


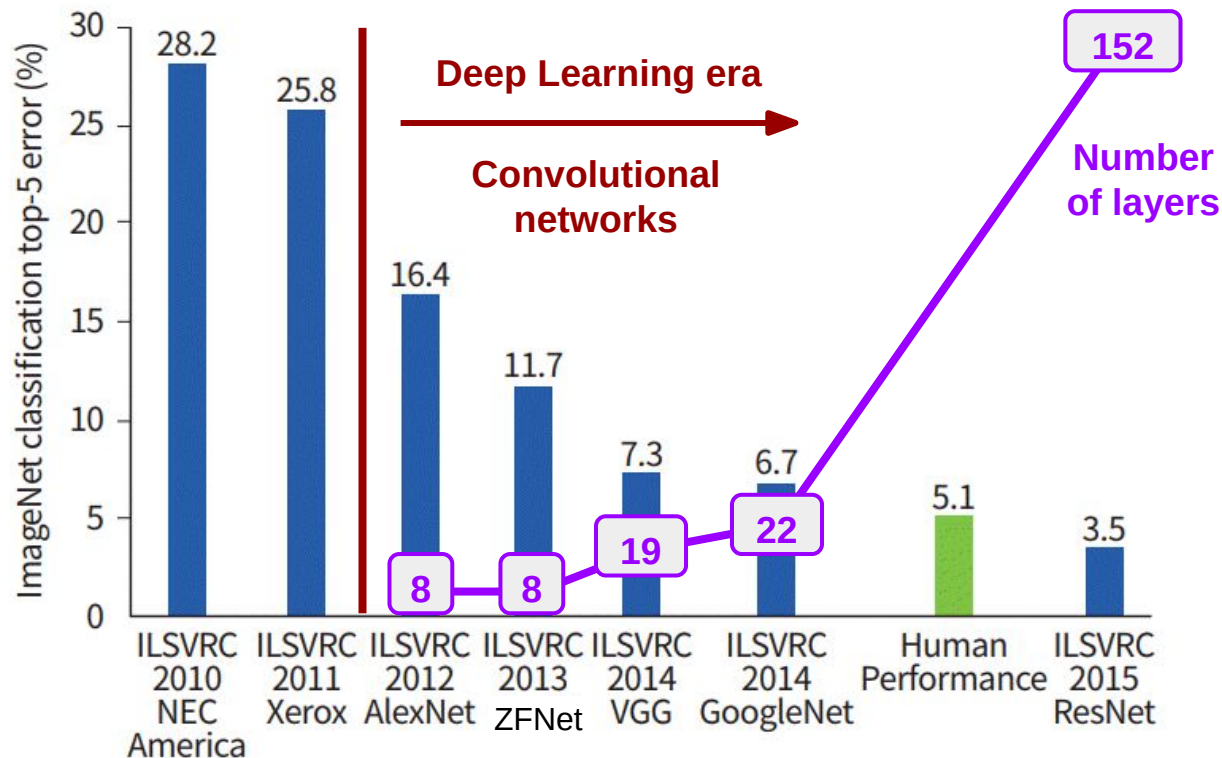
Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

- 1000 classes
- ILSVRC-2012: 1.2M images (~150Gb)
- Currently: >14M images in total
- Sped up development of deep learning

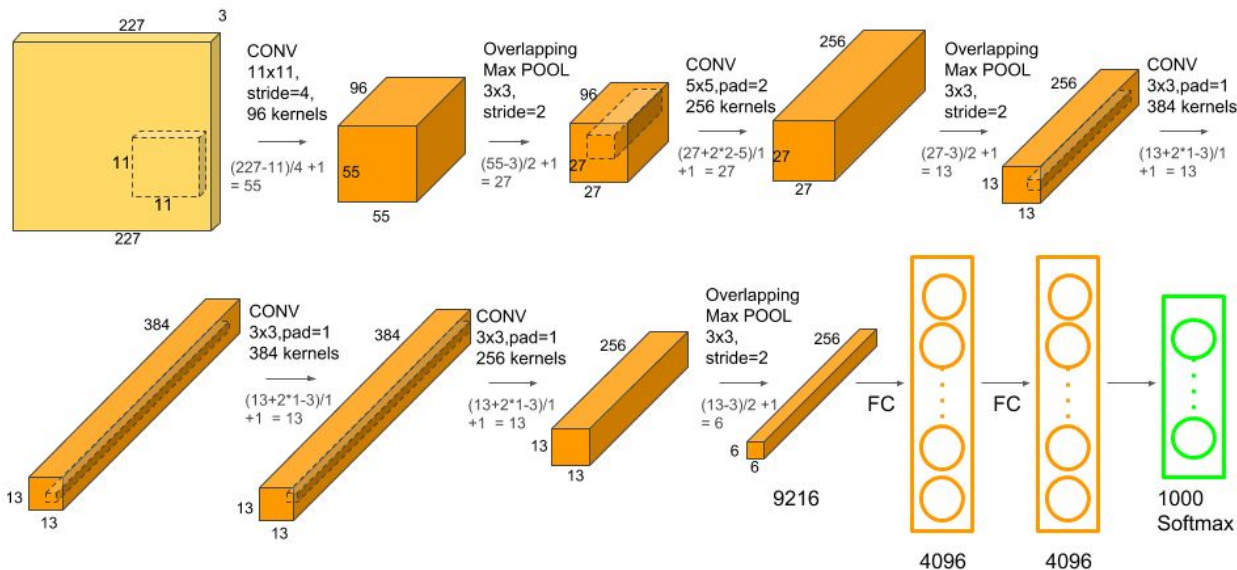


ImageNet Large Scale Visual Recognition Challenge (ILSVRC)



AlexNet (2012)

- Max pooling, ReLU non-linearity
- Dropout and Image augmentations to reduce overfitting



AlexNet		
Top-1 acc	Top-5 acc	#params
56.5	79.0	61.1M

**Tables taken from torchvision models*

Image augmentations

Original image



augmentation



Horizontal Flip



Crop



Median Blur



Contrast



Hue / Saturation / Value



Gamma



ImageNet augmentations

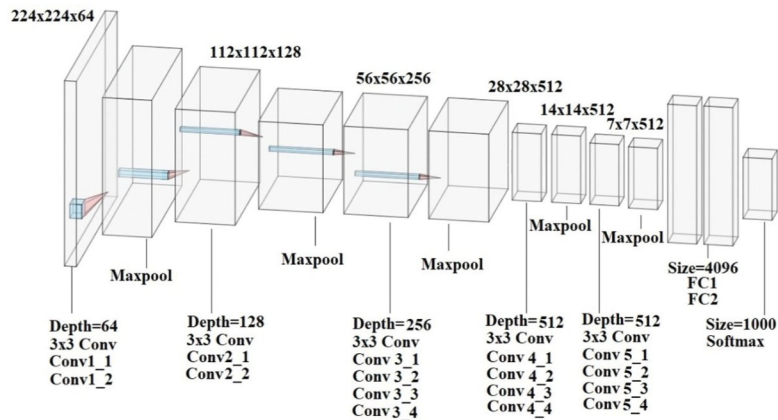
```
import torchvision.transforms as T

train_transform = T.Compose([
    T.RandomResizedCrop(224),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225])
])

test_transform = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225])
])
```

VGG (2014)

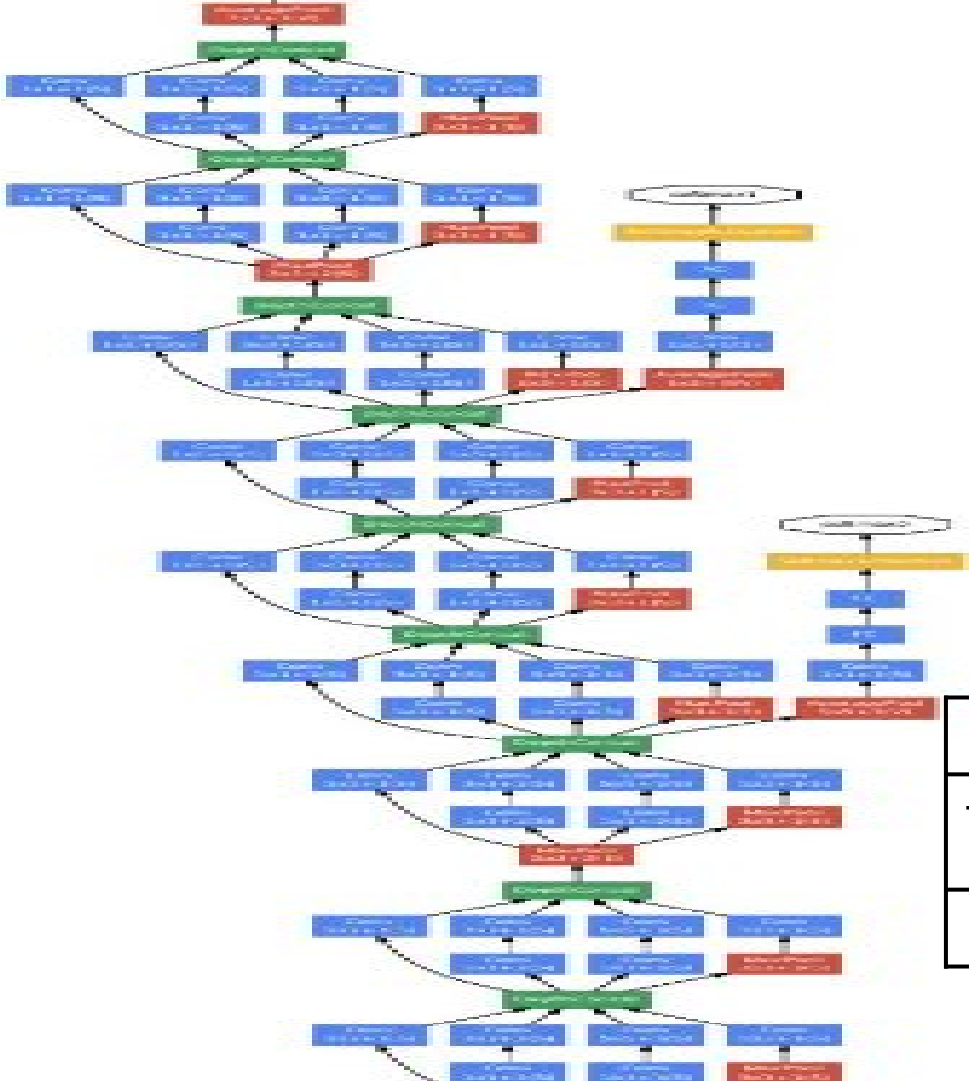
- Visual Geometry Group (University of Oxford)
- Deeper than AlexNet (16 or 19 layers)
- Smaller convolutional kernels (3 x 3), more balanced computations
- Does not train end-to-end – vanishing gradients
- Trained in a few stages with increasing depth
- 2 – 3 weeks on 4 NVIDIA Titan Black GPU



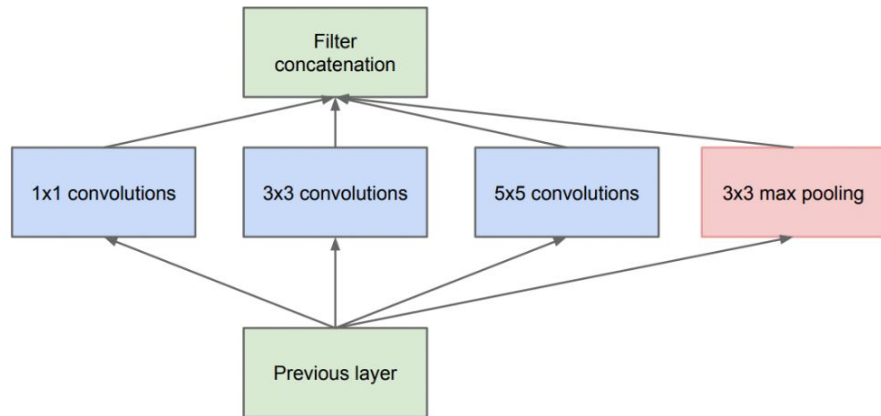
VGG-19		
Top-1 acc	Top-5 acc	#params
72.4	90.9	143.7M

Goog

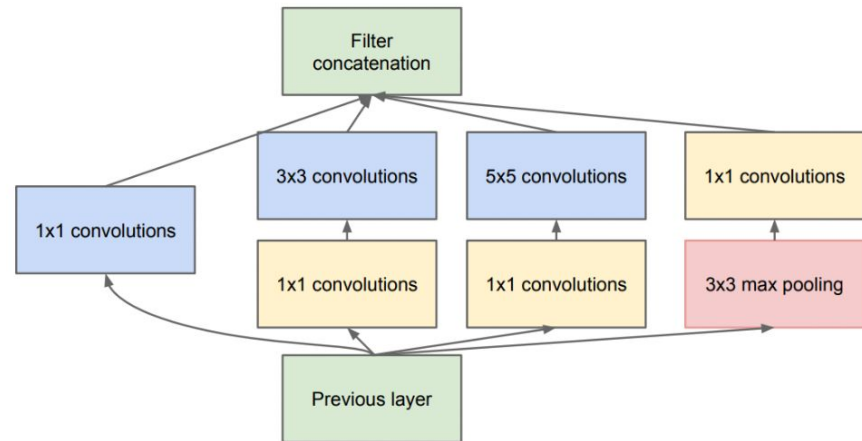
- F
- L



GoogLeNet		
Top-1 acc	Top-5 acc	#params
69.8	89.5	6.6M



(a) Inception module, naïve version

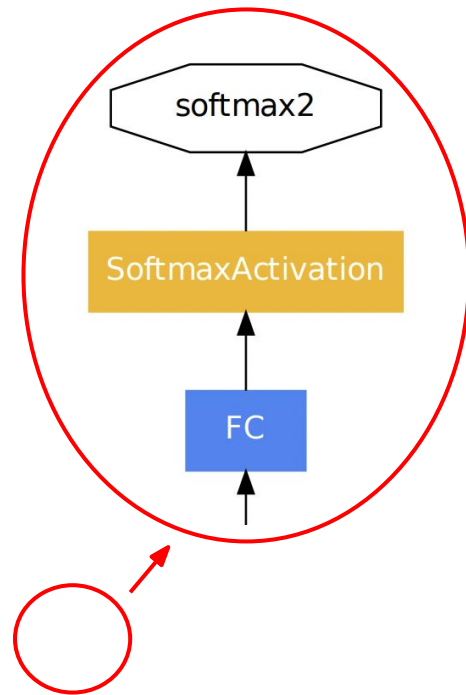


(b) Inception module with dimension reductions

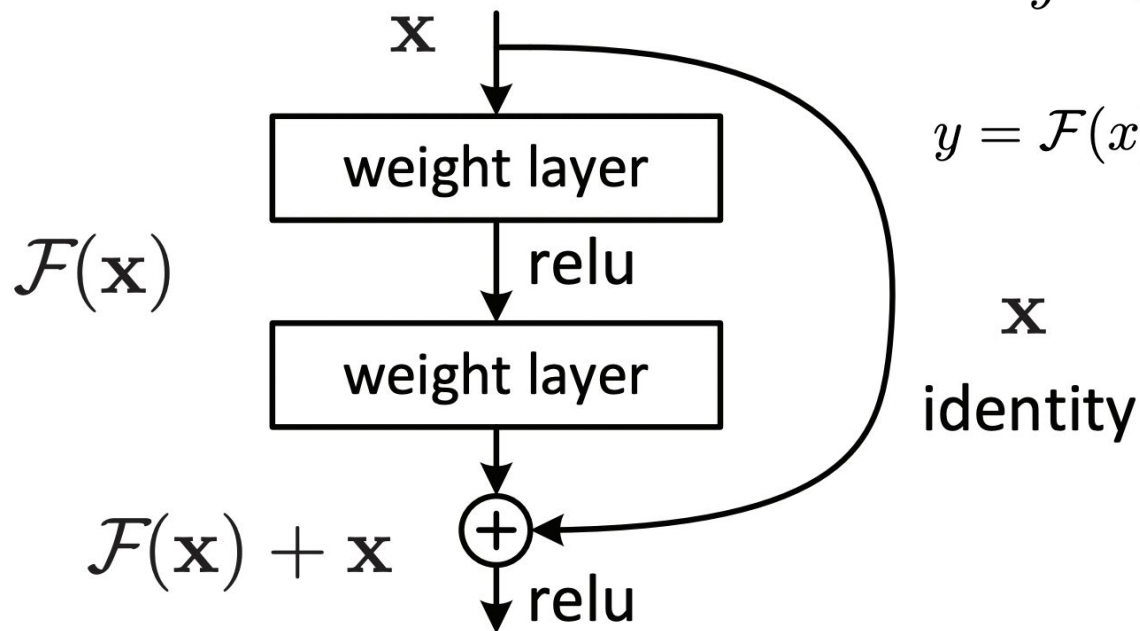
reduce number of channels with 1x1 convolutions =>
reduce computation time and memory consumption

GoogLeNet (a.k.a Inception, 2014)

- Does not train end-to-end, needs **auxiliary classifiers** to propagate gradient



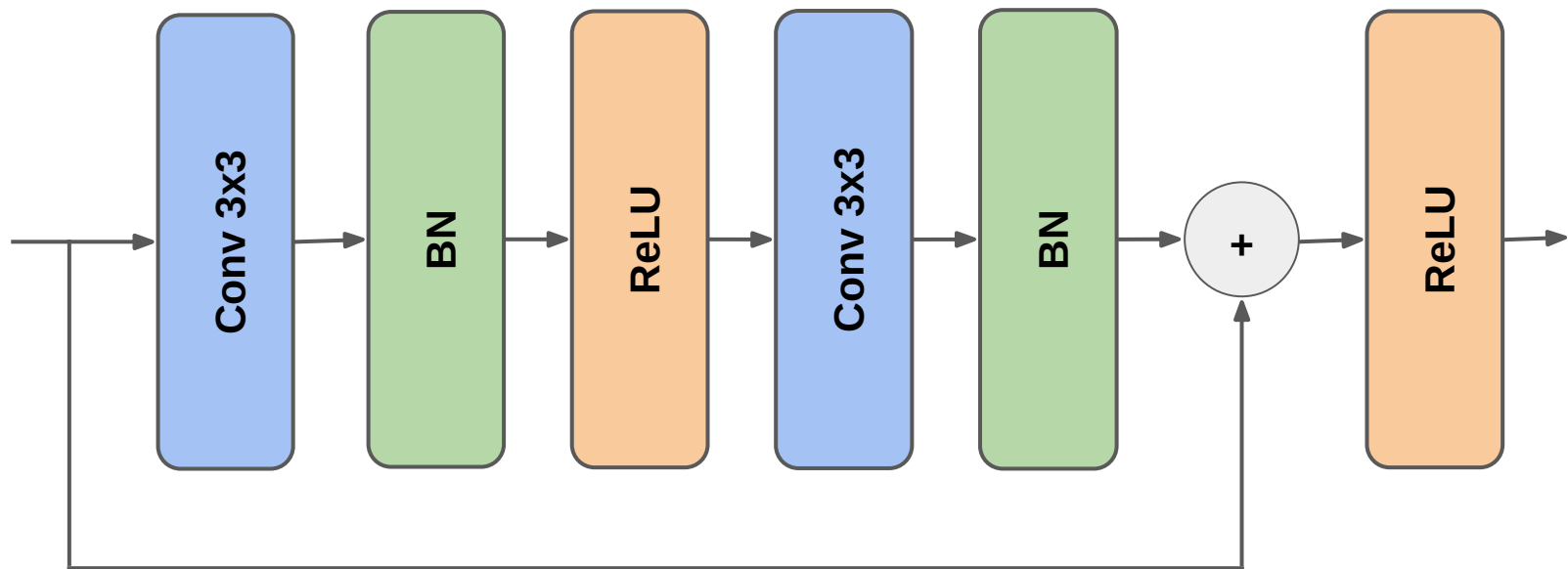
Residual (skip) connections



$$y = \mathcal{F}(x) \Rightarrow \frac{d\ell}{dx} = \frac{d\ell}{dy} \frac{d\mathcal{F}}{dx}$$

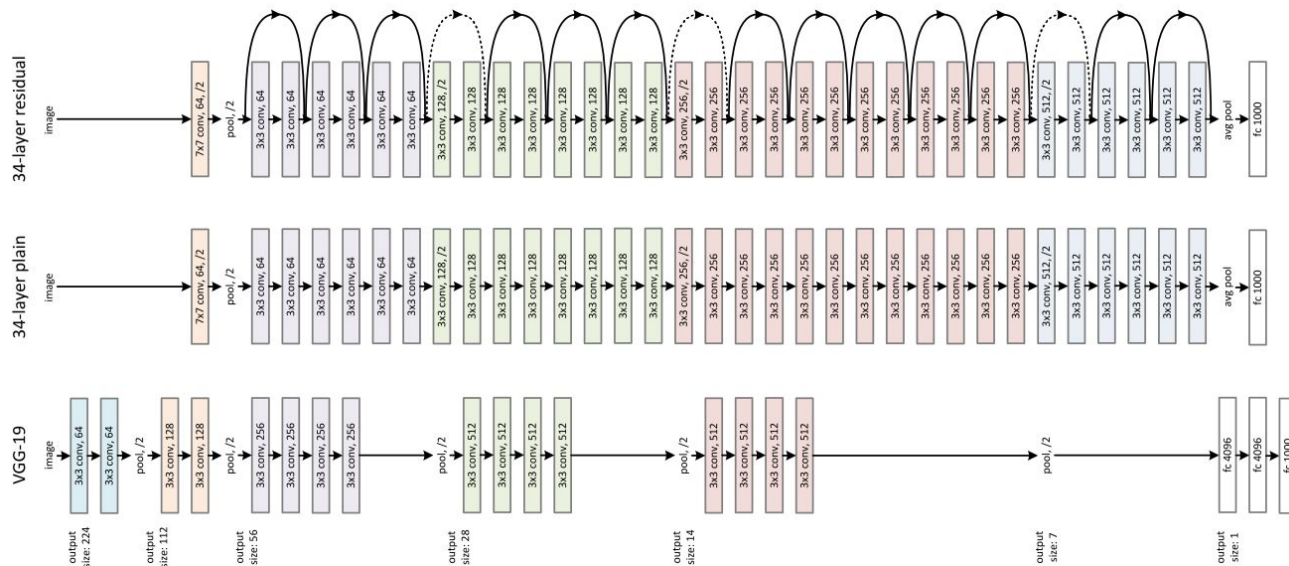
$$y = \mathcal{F}(x) + x \Rightarrow \frac{d\ell}{dx} = \frac{d\ell}{dy} \frac{d\mathcal{F}}{dx} + \frac{d\ell}{dy}$$

Residual block



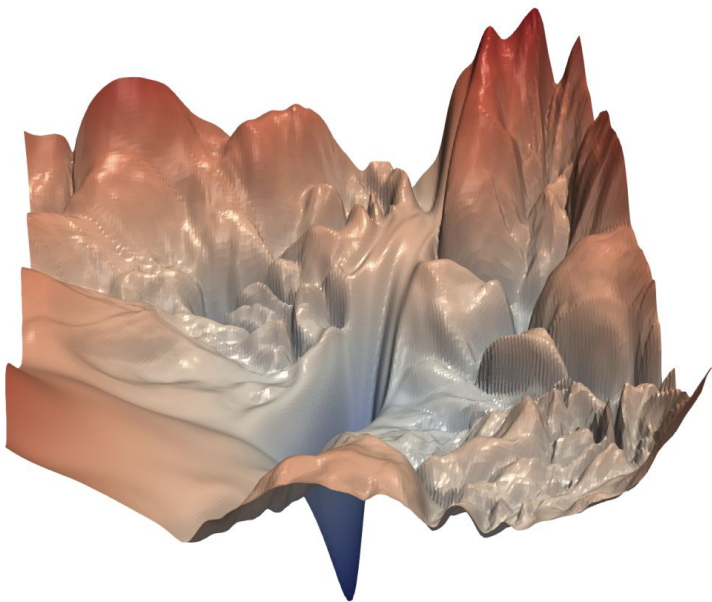
ResNet (2015)

- Residual connections make gradient flow better => stack **MUCH MORE** layers
- Batch Normalization** to stabilize training
- No Max pooling** in blocks, strided convolutions instead
- Global average pooling** of the resulting feature map => process arbitrary size images

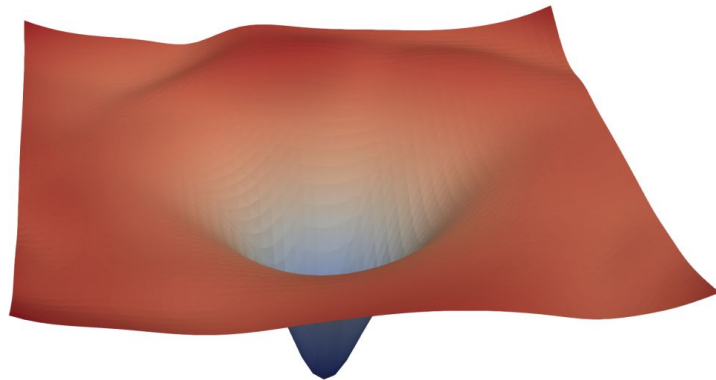


ResNet-152		
Top-1 acc	Top-5 acc	#params
78.3	94.0	60.2M

Why skip connections?



(a) without skip connections

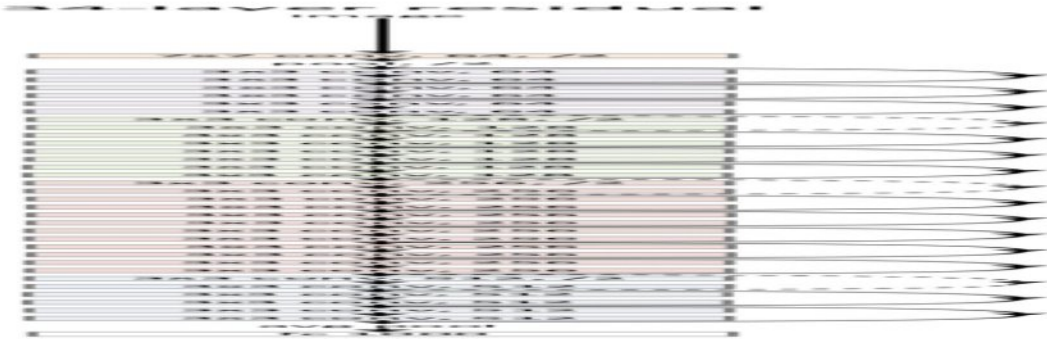


(b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

ResNet legacy

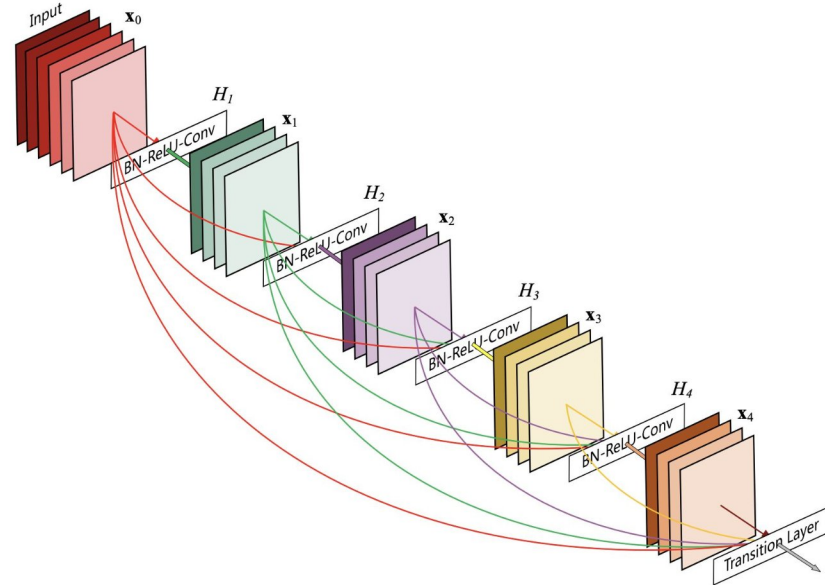
- All modern architectures use residual connections (even non-convolutional)
- ResNet direct successors:
 - **WideResNet** ([Zagoruyko and Komodakis, 2016](#))
 - **ResNeXt** ([Xie et al., 2016](#))
 - **Pre-activation ResNet** ([He et al., 2016](#))
- ResNet-18/34/50 networks are the most frequent CNN baselines



DenseNet

- Any 2 layers are connected
- Channel-wise feature map concatenation
- Very narrow layers (i.e. small number of channels)
- Less parameters than in ResNet

DenseNet-201		
Top-1 acc	Top-5 acc	#params
76.9	93.4	20.0M



MobileNet

- Lightweight architecture to be used on mobile devices
- Uses combination of depthwise and pointwise convolution instead of a regular one

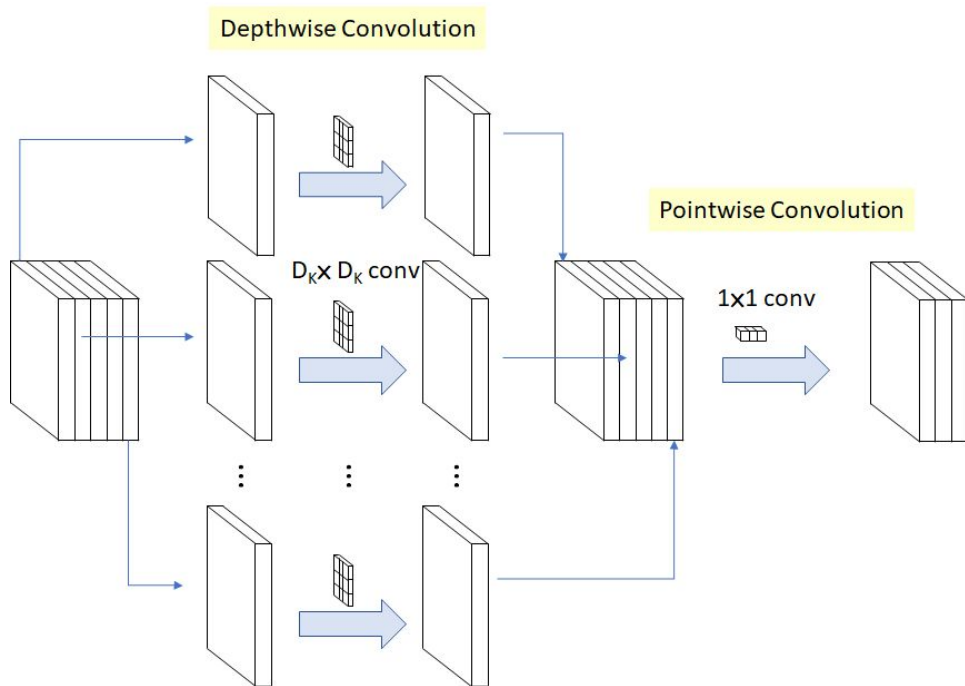


Table 4. Depthwise Separable vs Full Convolution MobileNet

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
Conv MobileNet	71.7%	4866	29.3
MobileNet	70.6%	569	4.2

MobileNet-v1		
Top-1 acc	Top-5 acc	#params
70.9	89.9	4.2M

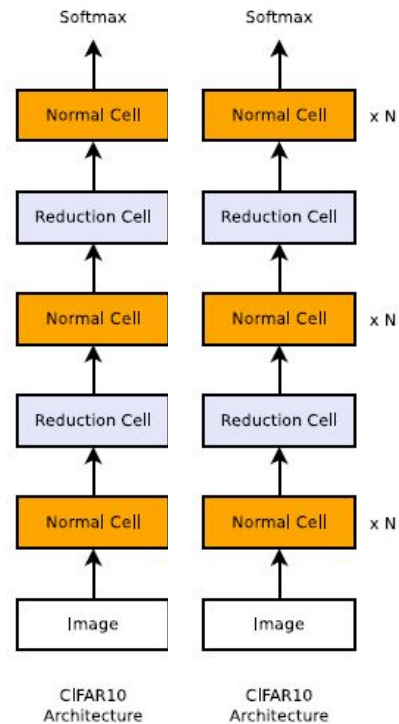
**Table taken from tensorflow models*

NASNet

- Neural Architecture Search (**NAS**) – meta-optimization of convolutional architecture
- Optimize architecture for small dataset (**CIFAR-10**), the transfer to large dataset (**ImageNet**)
- Search for optimal structure of **Normal cell** and **Reduction Cell**
- Use so-called **Controller Network** to predict cells structure trained using **Reinforcement Learning**

NasNet-A		
Top-1 acc	Top-5 acc	#params
74.0	91.6	5.3M

**Table taken from tensorflow models*

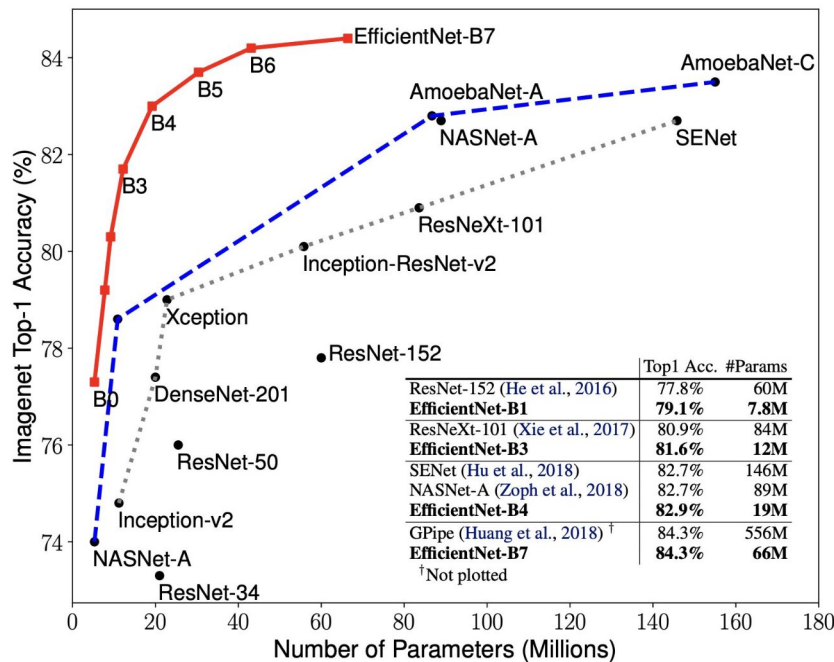


[Zoph et al., 2017](#)

EfficientNet

- Based on **MNAS** (mobile NAS)
- Finding optimal scaling of neural network **width**, **depth** and **image resolution** for different FLOPS budget

EfficientNet-B0		
Top-1 acc	Top-5 acc	#params
77.7	93.5	5.3M



Non-convolutional architectures

- Current **SOTA** (state-of-the-art) models are non-convolutional architectures
- Today's best models are usually **transformers** (to be discussed later)
- Convolutions are still frequently used
- CNNs are still popular, being faster and easier to train



What is next?

- **January, 2022** – new promising convolutional architecture, “A ConvNet for the 2020s”
- Main idea: mimic internal structure of transformer

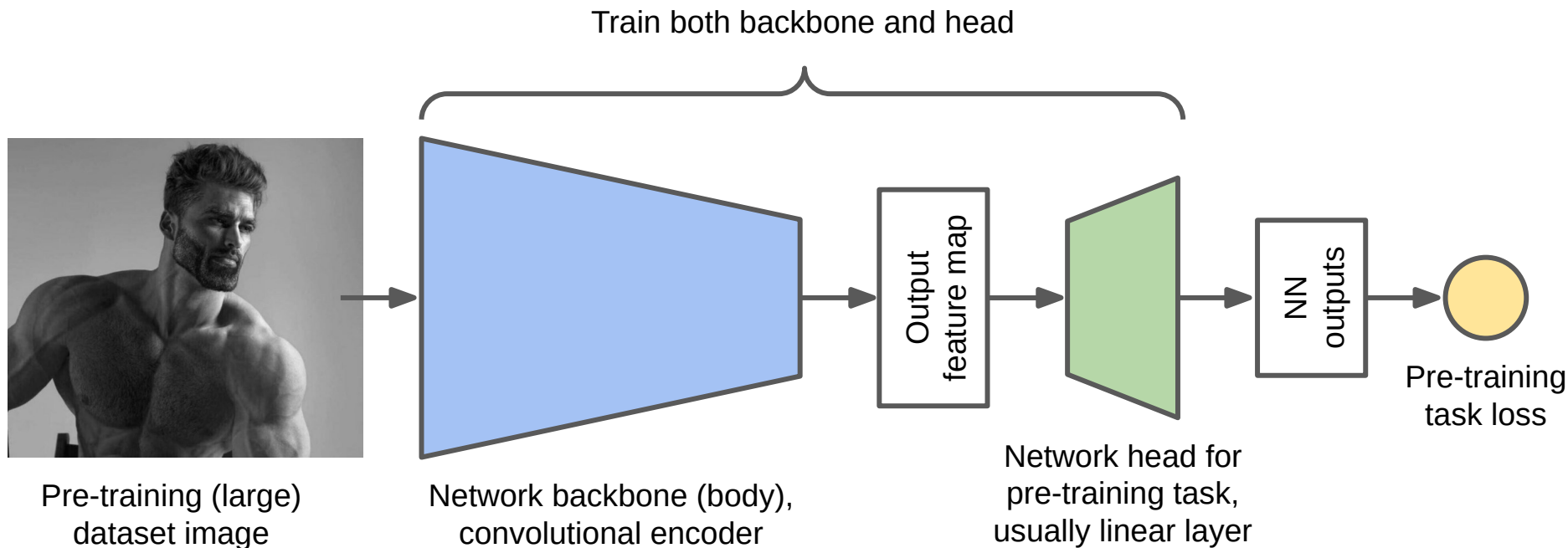


ConvNeXt-Base		
Top-1 acc	Top-5 acc	#params
84.0	96.9	88.6M

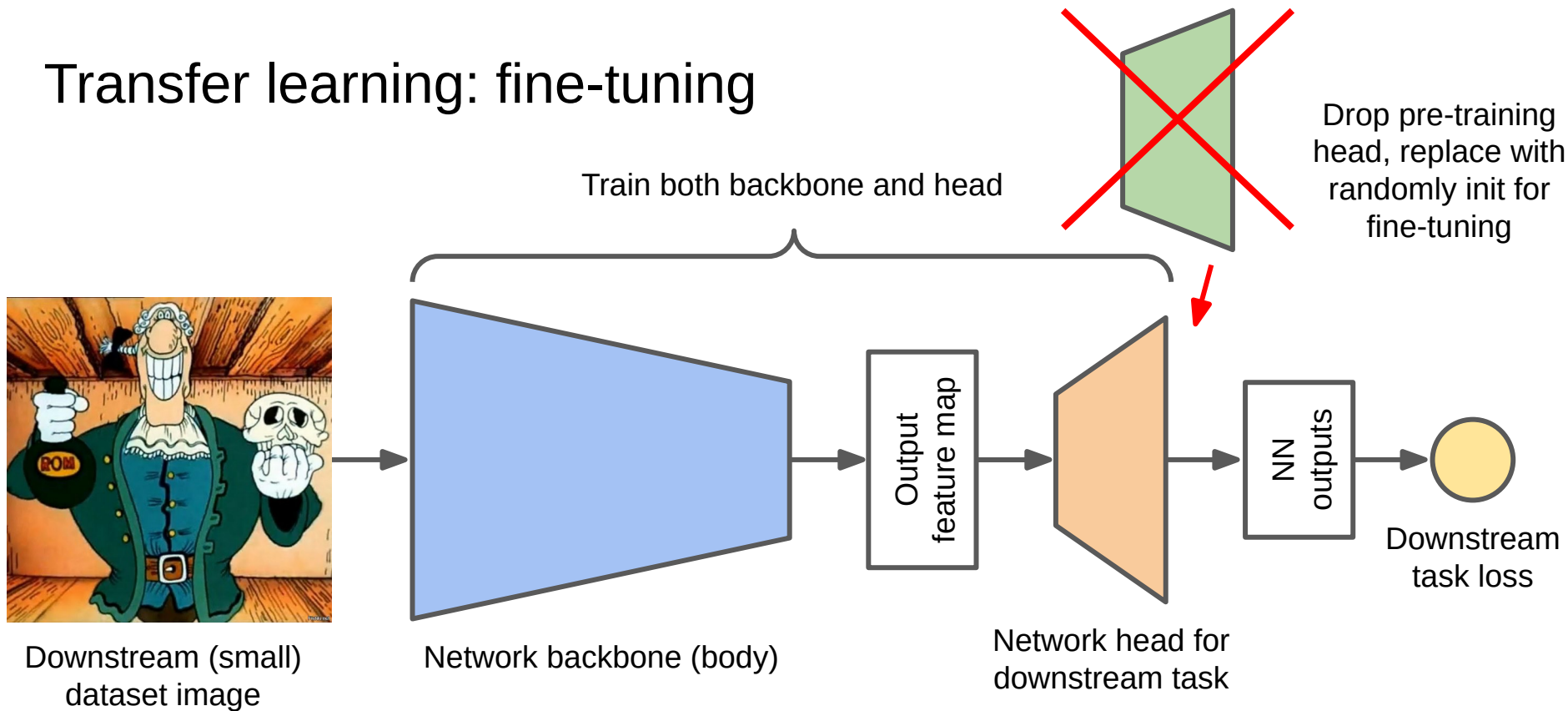
How to deal with small datasets?



Transfer learning: pre-training



Transfer learning: fine-tuning

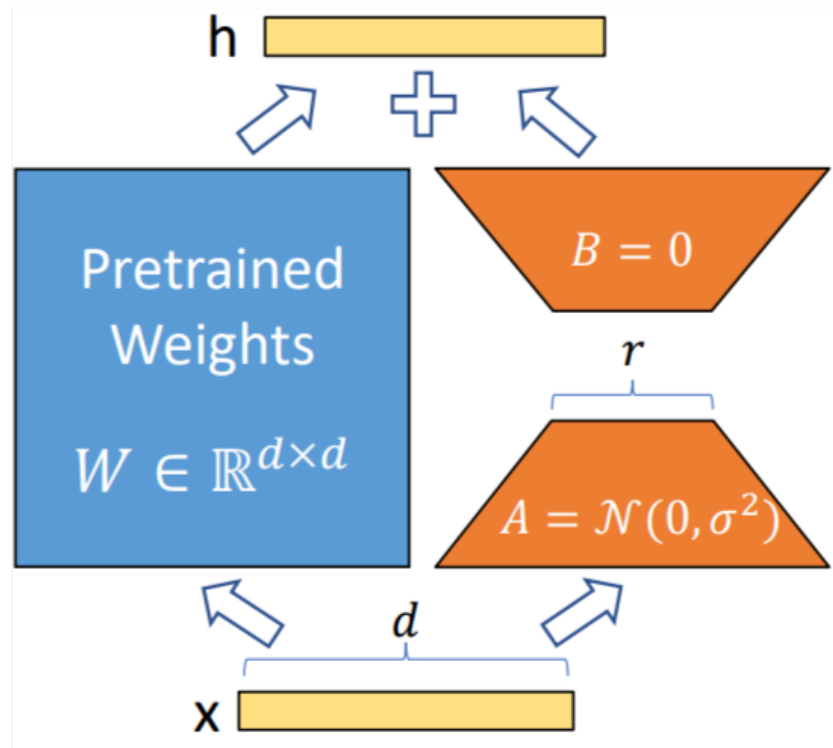


Transfer learning: LoRA

$$\hat{B} \in \mathbb{R}^{d \times r}, \hat{A} \in \mathbb{R}^{r \times \tilde{k}}$$

$$r \ll \min(d, k)$$

$$h = W_0 x + \Delta W x = W_0 x + B A x$$



Insert a low-rank addition into every linear layer

Transfer learning

- Main idea: re-use **NN weights** or **feature maps** from different datasets
- **Pre-training**: train full network on large dataset
- **Fine-tuning (FT)**: train both new head and the backbone
- **Linear probing (LP)**: train new head on top of the frozen backbone
- **LP** is faster (train only a linear model), **FT** is better (generally)
- **LoRA**: low rank addition to the weights, for transformers
- It is possible to freeze part of backbone layers
- It is possible to pre-train without labels (self-supervised learning, to be discussed later)